

IN422 - TP2 - Structures de données: listes, arbres binaires, arbres binaires de recherche

Grenoble INP – Esisar – année 2025-26



Version du 20 octobre 2025



Objectif pédagogique

- Utiliser la gestion de mémoire automatique en C via l'usage d'un ramasse-miettes.
- Utiliser en C des structures de données persistantes.
- Implémenter du code sur et utilisant des structures de données en C à partir de la description abstraite (TAD) de la structure.
- Illustrer l'usage de telles structures via des applications d'exemple



Evaluation

Chaque binôme rendra:

- une archive contenant le code réalisé;
- séparément de cette archive, un compte-rendu au format .pdf commentant les choix qui ont été faits, et contenant les analyses de vos expérimentations.

Il vous est demandé de produire vous-mêmes le code demandé, ainsi que le compte-rendu. Les emprunts aux collègues, et la délégation du travail à des IAs, ne vous permettront pas de récolter les bienfaits attendus: renforcer des savoirs, développer des savoir-faire. Dans le premier cas, il s'agit de plagiat, qui peut vous exposer à des sanctions.

Vous prendrez soin d'inclure vos deux patronymes dans chacun des deux fichiers. Les fichiers sont à déposer sur Chamilo, au plus tard le mardi 4 novembre.



Etape 0 : quelques explications

Dans tout ce TP, nous manipulerons des structures de données en laissant la gestion mémoire aux bons soins d'un gestionnaire automatique de mémoire fonctionnant selon le principe du *ramasse-miettes* (en anglais, *garbage collector*).

Le ramasse-miette utilisé se nomme tgc pour *tiny garbage collector* (<https://github.com/orangeduck/tgc>). Il consiste en deux fichiers `tgc.h` et `tgc.c`, présents dans le code fourni. Ensemble, ces deux fichiers font à peine plus de 500 lignes de code C.

L'utilisation de ce ramasse-miettes impose quelques règles et contraintes:

- Les allocations doivent se faire en utilisant le gestionnaire mémoire associé au ramasse-miettes: dans le code fourni, la mémoire dynamiquement allouée l'est toujours via un appel à la fonction `tgc_alloc()`.
- Le gestionnaire de mémoire doit être initialisé; c'est l'objet de l'appel `tgc_start()` que vous localiserez dans le `main()` de certains programmes fournis.

- Une contrainte d'utilisation pour ce gestionnaire est que les appels à l'allocateur de mémoire doivent se situer "un niveau plus bas" que l'adresse de référence utilisée dans l'initialisation de l'allocateur. Dans l'exécutable fourni, cela est réalisé en appelant une fonction depuis `main()`, et en laissant le soin à cette fonction où celles qu'elle appellent de faire tout le travail, et donc en particulier les allocations.



Etape 1 : prise en main du code fourni

- Récupérer l'archive fournie, et la désarchiver (commande `tar`).
- Observer le code et le `Makefile` fournis (rapidement, pour vérifier qu'il ne s'agit pas de logiciel malveillant).
- Compiler avec la commande `make`. Comprendre le `Makefile` fourni.
- Lancer l'exécutable de test de la liste chaînée, `t_liste_chaine`. Que fait-il?



Etape 2 : appropriation de l'implémentation de la liste chaînée.

Vous remarquerez que le module `liste_chaine.h/liste_chaine.c` implémente fidèlement le type abstrait de données (TAD) "liste chaînée" du cours. Dans la suite, les listes seront toujours manipulées à partir des 5 routines primitives fournies dans `liste_chaine.h`, ou en utilisant des fonctions écrites à partir de ces primitives.

En faisant référence à ces 5 primitives, nous pouvons écrire du pseudo-code qui manipule des listes chaînées.

- Complétez le module `fonctions_listes.c` avec les définitions des fonctions:
 - `liste_longueur()`
 - `liste_retourne()`
 - `liste_concatene()`

Testez leur bonne fonctionnalité en enrichissant le code de `t_liste_chaine`.



Etape 3 : une application concrète des listes: le tri fusion

Vous êtes maintenant prêts pour l'écriture de votre premier programme utile manipulant des listes.

Ecrire un nouveau programme C, nommé `fusion.c`, qui implémente le tri fusion. Votre routine `liste* tri_fusion(liste *l)` doit trier une liste d'entiers. Vous pourrez utiliser les routines du module `fonctions_listes.h`.

Evaluez la complexité en opérations de cette implémentation. Est-elle conforme à ce que prévoit la théorie?



Etape 4 : listes de listes

Copier le fichier `liste_chaine.c` en un fichier `liste_chaine_p.c`. L'objectif est de transformer le premier, qui manipule des listes chaînées d'entier, en un module capable de manipuler des listes chaînées de pointeurs. Testez la bonne fonctionnalité du module, par exemple en manipulant une liste de liste d'entiers.

Note: le fichier `liste_chaine_p.h` vous est fourni.



Etape 5 : énumération des parties d'un ensemble

Ecrire un nouveau programme C, `parties.c`, qui intègre une fonction `liste_p* parties(liste *l)`, qui retourne une liste de listes, la liste des parties de l'ensemble encodé par la liste initiale `l`.

Vous pourrez utiliser une fonction auxiliaire `liste_p* parties_aux(liste_p* accumulateur, liste *partielle, liste *restants)` qui reçoit une liste de parties déjà calculées (accumulateur), une liste partiellement formée, et une liste d'éléments restants parmi lesquels piocher d'autres éléments à rajouter (ou pas!) à la liste partielle.

Cette fonction peut être codée de manière récursive selon le principe suivant: s'il n'y a pas d'élément restant, on rajoute la liste partielle courante à la liste déjà accumulée; s'il reste des éléments, on fait deux appels récursifs: un appel où le prochain élément restant est ajouté à la liste partielle, et un appel où il n'est pas ajouté.

Testez votre fonction sur une liste à 3 éléments, et sur une liste à 4 éléments.



Etape 6 : partage mémoire dans les structures de données persistantes

Copier le code de la fonction `liste_affiche(liste *l)` pour en faire une fonction:

```
liste_affiche_adresses(liste_p *l)
```

qui affiche les adresses des éléments (de type `(void*)`) de la liste. Vous pourrez utiliser la chaîne de formatage `%p` de la fonction `printf()`.

Affichez les adresses du résultat retourné par `parties()` sur une liste à 3 éléments. Combien de cellules de listes ont été allouées lors de ce calcul?



Etape 7 : 'Subset sum': problème du sous-ensemble de somme donnée

Ecrire une fonction `int liste_somme(liste *l)` qui calcule la somme des entiers d'une liste. Utiliser cette fonction pour écrire une fonction `liste* subset_sum(liste *l, int n)` qui trouve, lorsque cela existe, un sous-ensemble des éléments de la liste `l` dont la somme fait exactement l'entier `n` donné. Elle retournera la liste vide lorsqu'il n'existe pas de telle liste.

Illustrez l'usage de cette fonction en calculant lesquels des nombres entre 100 et 150 peuvent être écrits comme somme de nombres parmi 3, 6, 7, 21, 43, 51, 126.